

Modeling Strategies for Horse-Racing Betting Exchanges (Betfair)

Feature Engineering from Price and Order-Book Data

Successful models exploit **exchange market microstructure data** beyond just final odds. This includes real-time **prices, volumes, and order book** metrics that capture supply-demand imbalances:

- **Best Back and Lay Prices & Spread:** The core snapshot features are the current best available back price and lay price for each horse, and the *spread* between them. A tight **market spread** (small gap) often indicates an efficient market, whereas a wide spread shows uncertainty or low liquidity. For example, one Betfair analysis defines the spread as the number of **tick intervals** between the best back and best lay; as race time approaches, spreads tend to shrink (market “tightens”) ¹. The market overround (sum of implied probabilities) likewise converges toward 100% near the start ² – early on you might see back odds summing to 108% vs lay odds 81%, which gradually closes up, indicating increasing confidence and liquidity.
- **Market Depth and Liquidity Imbalance:** Beyond the best prices, consider the **depth** of the order book – e.g. total volume available at the top 3 back prices vs top 3 lay prices. A **liquidity imbalance** (significantly more money on one side) can signal likely price movement as the market corrects. For instance, if far more volume is queued to back a horse than to lay it, the price may shorten (the weight of money effect). *Available volume* at best prices and cumulative **traded volume** are very actionable features. It’s known that the bulk of trading happens in the last minutes before a race – one study showed total matched volume roughly doubles from 10 minutes out to 4 minutes out, and then doubles again by 1 minute out ³. This late surge means **time-to-off** is critical context: features like “volume traded in last 60 seconds” or “price trend in last 5 minutes” often have predictive power, since early price movements with low volume can be misleading.
- **Price Drift and Volatility Features:** Horse racing markets exhibit “**steamers**” (odds shortening rapidly) and “**drifters**” (odds lengthening) as information flows. It’s useful to quantify a runner’s recent price change – e.g. the percentage change in odds over the last X minutes or the slope of odds movement. Sudden large moves, especially close to the off, may indicate **informed money**. For example, if a horse’s odds drop by 12% in the final 15 minutes with no new public information, that could be a strong signal (a case where “smart money” is causing a price drift) ⁴. Features capturing **momentum** (direction of price change) or **volatility** (frequency and magnitude of price updates) can help a model distinguish normal market noise from meaningful moves. Similarly, the **imbalance of trades** (more aggressive back bets vs lay bets in short succession) is a predictive signal – effectively a proxy for bullish or bearish sentiment on that runner.
- **Relative Price Metrics:** Because Betfair odds are on a nonlinear scale (1.01 to 1000 with varying tick sizes), converting prices to a **normalized scale** can help. Some practitioners use the **Betfair tick index** – essentially ranking the price on the predefined odds ladder ⁵ – to measure movements in

consistent units. This allows features like “odds change in ticks per minute” regardless of whether a horse is at 3.0 or 30.0 (since a 0.1 move at low odds is more significant than the same move at high odds). Additionally, comparing a runner’s current implied probability to its **Betfair Starting Price (BSP)** implied probability can flag if it’s drifting beyond expectation. (However, note that BSP itself isn’t available until the off – using it in training must be done carefully to avoid leakage).

In summary, engineering features that capture **market state (spread, overround)** ¹, **liquidity (volume distribution)** ³, and **price dynamics (recent drift/volatility)** will arm the model with insights similar to what a seasoned trader monitors. These features help the model detect late momentum or imbalances that the raw odds alone might not reveal. Always be mindful of timing – e.g. a price 10 minutes ago is useful, but its relevance decays as the race draws near, so time-weighted features or windowed statistics are valuable.

Target Formulations: Outcome vs. Price Movement vs. Value Classification

Choosing the right prediction target is a fundamental design decision and should align with your **strategy objective** (predicting winners vs trading on movements vs finding mispriced bets):

- **Win Probability (Outcome Modeling):** A classic approach is to predict the probability of each horse winning the race. This turns the task into a **probabilistic classification** (or regression to a [0,1] probability) for each runner. If well-calibrated, these probabilities can be directly used to identify **value bets** – cases where the model’s estimated win probability is higher than the implied probability from the market odds ⁶. For example, if a horse is 5.0 odds (20% implied chance) but your model gives it a 25% win chance, that’s an overlay (value bet). The advantage of this target is that it directly models the end outcome needed for profit; it’s suitable for **finding edges against the BSP or bookmaker odds**. However, the challenge is extreme class imbalance (only 1 horse wins out of, say, 12 – more on that in the imbalance section) and the fact that the true outcome is only known after the race. Many machine learning applications in sports betting focus on outcome probabilities, but it’s crucial to emphasize **calibration** – a slightly less accurate but better-calibrated model is preferable, since profitable betting requires that a “20% probability” truly wins ~20% of the time ⁷ ⁸. In practice, you might train a model to minimize log-loss or Brier score so that these win probabilities are as accurate as possible.
- **Price Movement Prediction:** In a betting exchange (especially pre-race trading), one might instead target **short-term price changes**. The model tries to forecast if the odds will move **up or down** (or by how much) in the next interval (e.g. next 30 seconds or 1 minute). This is analogous to predicting stock price movements and is useful for **trading strategies** (e.g. bet now and lay off later for a profit). Recent research has explored this approach; for instance, a 2025 study applied deep convolutional attention networks to **forecast Betfair price movements in the short term** using market depth data ⁹. Typically this is set up as a classification (will the price go *down* = horse’s probability improving, or *up* = drifting, or no change) or regression (predict the next price or delta). The training labels can be derived from tick data: e.g. if the odds 10 seconds later are lower than now, label = “price down”. This target leverages high-frequency patterns and order book dynamics – essentially capturing **momentum trading signals**. The key is defining a sensible horizon (too short and it’s mostly noise, too long and many unpredictable events can intervene). **Intermediate targets** like “will this horse’s odds drop by at least 5 ticks before the off?” are also used – essentially a

classification of a significant price move. The advantage here is more *immediate feedback* for the model (many more examples of price moves than outright wins), and it aligns with how traders operate (entering and exiting positions based on price action). If you go this route, ensure your validation simulates live trading (since you'll be using these predictions to trigger trades).

- **Value/Edge Classification:** Another formulation is a two-tier approach: first compute features relating to whether a bet has **positive expected value**, and then directly **classify bet opportunities as “value” or not**. This differs from pure outcome modeling – it uses the market odds in the target definition. For example, you could label a training instance as *1 (value bet)* if placing a bet at the available odds would have yielded a positive net profit in the long run (this can be approximated by whether the horse won *and* the model thought it had >X% chance, or by comparing model probability to market probability with a threshold). In effect, the model learns to predict “**was this bet +EV?**”. One way to construct such labels historically is: use your model's probability (or an ensemble of models or hindsight data) as an oracle and mark when the market odds were an overlay. However, caution is needed to avoid circular logic (using the same model to label and predict). A more robust method is to use **cross-validation**: for each race, use probabilities from an out-of-sample model to decide if a value bet existed, then train a secondary model on those labels. This approach was hinted at by practitioners who “recalibrate value thresholds” – essentially training a filter for when to bet ¹⁰. The benefit of value classification is that it directly optimizes what you care about (making profitable bets), rather than accuracy of win picks. It can also incorporate factors beyond pure win probability – e.g. the model might learn that certain races (or certain odds ranges) are not worth betting even if an edge appears, due to higher uncertainty or market volatility (thus serving as a guardrail).

In practice, you might combine these formulations. For instance, a common setup is: **Model A** predicts win probability for every horse (giving you an initial sense of value against the odds), and **Model B** acts as a decision layer, perhaps predicting whether the **probability advantage** of Model A is trustworthy enough to bet (a form of gating). This two-stage approach leverages the strengths of both: Model A sees the broad picture (all runners' chances), Model B can account for conditions that affect betting outcomes (like market conditions, or “does this race fit a pattern where the favorite is often underpriced?”). Indeed, research shows that blending model predictions with market data yields better results – for example, a team improved profit by **stacking their model on top of the market odds** themselves ¹¹. They essentially used the **Betfair Starting Price** as a feature alongside model outputs to predict outcomes, acknowledging that the crowd wisdom is a strong predictor and the model's job is to make slight improvements on it. Ultimately, define your target to match your betting strategy: if you aim to pick winners to bet fixed odds, focus on win probability calibration; if you aim to trade (buy low, sell high), focus on short-term price direction; if you aim to deploy capital only when an edge is clear, consider an explicit value classification stage.

Time-Aware Validation and Probability Calibration

When modeling betting data, **time-aware validation** is critical to obtain realistic performance estimates. Races are chronological events, and markets evolve over time, so your training/validation scheme must respect that order to avoid lookahead bias (**data leakage**).

- **Rolling/Expanding Window Cross-Validation:** Instead of random splits, use a **walk-forward validation** approach. For example, train your model on historical data up to 2023, and validate on races in early 2024; then roll forward – include early 2024 in training and test on later 2024, and so

on. This simulates how you'd continually retrain as new data arrives ¹². Such **rolling windows** (or an expanding window that keeps growing) ensure that at each test point, the model has only seen the past, never the future. This method is "**evaluation on a rolling forecasting origin**" – essentially repeated train/test splits that move chronologically ¹³. It not only prevents leakage of future information but also helps capture regime changes (maybe the betting market dynamics in 2021–2022 differ from 2018–2019, for instance). A concrete example for a validation fold: *Train on races Weeks 1–8, test on Weeks 9–12; then retrain on Weeks 1–12, test on 13–16*, etc. ¹². This approach was explicitly recommended for sports betting models to mimic real-time betting conditions ¹⁴.

- **Avoiding Leakage: Data leakage** can ruin a model's performance by introducing information that wouldn't be available in a live scenario. Common pitfalls include: using final race outcomes or final odds in features for earlier times, or mixing data from the future in training. For instance, including a horse's Betfair Starting Price (BSP) as a feature for pre-off prediction is a leak – BSP is determined at the moment the race starts (when betting ends) and obviously wouldn't be known an hour before the race. In a published horse racing model study, the authors admitted a caveat that their model inadvertently used BSP as a feature, which "**is only available after the race has started**", i.e. not a truly prescient feature ¹⁵. During validation, one must ensure such info isn't sneaking in. Another example: if modeling live in-play odds, you must not feed an event (like a goal in soccer, or a horse falling) into the model before it's happened in the timeline. From a workflow perspective, *partition data by time*. Also, do feature generation in a way that respects time (e.g. if creating a moving average of odds, only use past timestamps). A practical tip is to simulate the production environment when generating validation predictions: iterate through races chronologically and at each race, compute features only from that race's prior history. The **Betfair Data Scientists Hub** emphasizes "**seal leakages**: don't include data you wouldn't have known at the time of the bet" ¹⁶ – for example, if you build a model on Friday for Saturday's races, you shouldn't incorporate Saturday morning information (like a last-minute jockey change or a heavy rain announcement) if your model realistically wouldn't have it. Keeping the validation "pure" in time makes your performance estimates much more reliable.
- **Out-of-Fold Probability Calibration:** Calibration refers to making sure the predicted probabilities correspond to true outcome frequencies. In sports betting, **calibration is arguably more important than raw accuracy** ⁷, because we make money from probabilities being right, not just picking winners. A well-calibrated model means if it says a horse has a 10% chance, that horse really wins ~10% of the time in the long run. Techniques like **Platt scaling** (logistic regression on the model's output) or **isotonic regression** can adjust predicted probabilities using a hold-out set or cross-validation folds. The best practice is to perform calibration on **out-of-sample data** – for example, train your model, then apply it to validation races, collect those predictions and actual outcomes, and fit a calibration curve (like an isotonic mapper) on that. This calibrated model can then be used on new predictions. You can even integrate this into cross-validation: scikit-learn's `CalibratedClassifierCV` essentially does this by using fold predictions to fit calibration ¹⁷.
- **Measuring Calibration:** Always evaluate calibration on your validation data. Use a **reliability plot** (plot predicted probability vs actual win frequency). A simple diagnostic is to bin predictions into deciles (0–10%, 10–20%, ... 90–100%) and check, for each bin, what fraction actually won. For example, if in all races where your model gave 40–50% win probability, the horse ended up winning about 45% of the time, that bin is well-calibrated ¹⁸. If you see your 80% predictions only winning 60% of the time, the model is overconfident (needs calibration). Metrics like **Brier Score** (mean

squared error of the probability forecast) and **Log Loss** directly capture calibration quality – lower is better, and a well-calibrated model will achieve lower values by better reflecting uncertainty ¹⁹. One 2024 study on NBA betting explicitly demonstrated that selecting models by **calibration (log loss)** led to far superior betting returns than selecting by accuracy, and noted “*Kelly betting only works with a well-calibrated model*” ²⁰. In our context, that means if you intend to size bets by predicted edge or use Kelly criterion, you must invest effort into calibration so that a 5% edge is truly a 5% edge on average.

- **Expanding Window for Calibration:** Remember that in a live setting, calibration may drift over time if the model’s performance changes. It’s wise to periodically recalibrate using recent data. For example, you might calibrate on the last 3 months of predictions (out-of-sample from training) to correct any mis-estimation. This can be folded into the rolling validation: each time you retrain on new data, also update the calibrator with the latest fold. The Betfair Hub content suggests plotting **decile reliability over time** as well, to ensure the model maintains good calibration as markets evolve ¹⁸.

In short, **treat time as sacrosanct in modeling and validation**. Use walk-forward splits, be vigilant about leakage (no peeking into the future), and put significant emphasis on calibrating probabilities out-of-sample. This yields models that you can trust in a live betting scenario, where good calibration can be the difference between a profitable Kelly betting strategy and a bankroll blow-up ²¹.

Handling Class Imbalance Without Breaking Probabilistic Meaning

Horse racing datasets are inherently imbalanced from a modeling perspective. In a race of 12 horses, the “win” class for a given horse is 1 and “lose” is 11 – only ~8% chance on average. Most horses lose, so a naive model could get 92% accuracy by always predicting no win, yet that’s useless. Similarly, if you frame a price movement prediction where “up” moves are rarer than “no change”, you face imbalance. Handling this imbalance is crucial, but **must be done in a way that doesn’t distort predicted probabilities** – because the *magnitude* of the probabilities matters for betting.

Common approaches include **class weighting** (telling the algorithm to pay more attention or give higher loss for the minority class) or **resampling** (oversampling wins or undersampling losses to balance the dataset). Algorithms like XGBoost and LightGBM offer a parameter `scale_pos_weight` (or an `is_unbalance` switch) to automatically weight the positive class. This can significantly improve the model’s ability to detect rare positives (e.g. longshot winners), *however*, it tends to **skew the probability outputs**. The LightGBM documentation explicitly warns that using `class_weight` or `scale_pos_weight` will “*result in poor estimates of the individual class probabilities*” and suggests performing probability calibration afterward if you use them ²². In other words, the model might still rank the horses correctly (so you’ll pick the right winner more often), but the raw predicted probability for each horse could be off – often **over-estimated** for the positive class due to the weighting ²³. A GitHub issue on XGBoost noted the same and even provided an equation to adjust the probabilities back after using `scale_pos_weight` ²⁴ ²⁵.

Best practices to handle imbalance in a betting context: - **Use Multi-class Modeling if Feasible:** Instead of building N separate binary models for N horses, you can build one model with N classes (one for each horse winning). This way, the probabilities for a given race are naturally constrained to sum to 1 (or 100%) across all horses. It inherently acknowledges that exactly one will win, which is additional structure for the model. Algorithms like neural networks or multinomial logistic regression (softmax output) or specialized

approaches like Plackett-Luce models use this setup. It avoids the imbalance problem altogether in a sense – you’re not dealing with a 1 vs 11 problem, you’re directly assigning win probabilities among the 12. The Betfair market odds essentially do this (they form an implicit probability distribution). If using tree-based models, one way to approximate this is to train a binary model but include **race-level normalization**: e.g. include as features some representation of the competition (like each horse’s market odds as features, or a “field strength” factor) so that the model “knows” not all horses can win. Some academic approaches (e.g. using a *Bradley-Terry* or Plackett-Luce formulation) embed that structure during training rather than after. In any case, a multi-class approach ensures probabilities stay coherent and usually mitigates the need for heavy class weighting.

- **Careful Weighting + Post-Calibration:** If you do use class weighting to improve recall of long-shot winners, plan to **recalibrate** the predictions. For example, you might set `scale_pos_weight` in XGBoost to, say, 10 (if winners are ~10x rarer than losers in the data) to force the model to pay attention to winner patterns. After training, collect the predicted probabilities on a validation set and apply a calibration fix (like solving for a logistic correction or using isotonic regression) to adjust these probabilities. There are known formulas to correct the bias introduced by weighting ²⁶. Even simpler, using an approach like **Platt scaling on validation** will inherently adjust for any bias. Essentially the model’s scores (logits) contain the rank information and some distorted scale; calibration can pull those back to proper probability scale. Empirically, some bettors find that disabling `scale_pos_weight` yields better calibration out-of-the-box ²⁷ – so one strategy is: train without weighting, but use other means (like more data or feature engineering) to handle imbalance, and rely on the model’s raw probabilities. If you do need weighting to get any signal on rare events, just remember the probabilities are *not directly usable* until adjusted.
- **Oversampling/Undersampling Strategies:** Another approach is to oversample the minority class (wins) or undersample majority (losses) during training. For instance, you could include the winning outcome multiple times or remove some losing outcomes to balance. This, like weighting, will affect probability outputs. If you oversample, your model might output probabilities assuming a higher base win rate than reality. So here too, calibration or manual adjustment of the output is needed. One could undersample such that, say, you keep all winners and randomly down-sample losers to a 1:4 ratio in each training batch. The model then doesn’t get overwhelmed by negatives. You must then interpret its output carefully – maybe calibrate by injecting the known base rate back in. Some practitioners use *threshold-moving* instead: accept that the model’s probability is off but set a higher threshold for betting to compensate (though a more principled way is explicit calibration).
- **Preserve Probability Meaning:** In all cases, we want the final probabilities to represent *true chances*. One way to check this in racing is to see if the probabilities for a race sum to around 100%. If you model each horse independently with a binary classifier, you might end up with predicted win probabilities that sum to 200% or more across the field – a sign of trouble. Calibration can bring this back in line (by effectively learning the appropriate “softmax” transformation). You may also incorporate a **global normalization** step: e.g. divide each horse’s predicted win odds by the sum of all horses’ predicted odds to force a sum of 1. This ensures a coherent probability distribution per race (though it doesn’t guarantee calibration in the absolute sense). The Betwise research team, for example, evaluated a **Plackett-Luce model** specifically because it produces a proper probability distribution over horses and found it a convenient way to ensure coherence ²⁸ ²⁹.

In summary, **address class imbalance by informing the model about true event frequencies, but correct for any bias this introduces**. The goal is to keep the **ranking performance** (ability to spot likely winners or price moves) while aligning the **probability scale** with reality. If you use boosting models, remember the mantra from their docs: *if you care about actual probabilities, either avoid heavy handed imbalance treatments or be ready to calibrate after* ²². A well-calibrated, probabilistically-sound model is indispensable for betting – you don't want to bet a horse thinking it's a 30% chance when it's actually 10%.

Strategy Evaluation: ROI, Turnover, Drawdown, and Execution Realism

After building a model and formulating a strategy, you need to **evaluate it rigorously** on historical data (and eventually in live betting) using appropriate metrics. Profit alone isn't a sufficient measure – you want to understand returns relative to risk, consistency, and whether the strategy holds up after real-world frictions like commission. Here are key evaluation dimensions and best practices:

- **Return on Investment (ROI):** ROI is the **primary metric for betting strategy performance**, typically defined as total profit divided by total amount staked. For example, if you bet \$1000 in total and end up with \$1,100, your profit is \$100 which is +10% ROI. A higher ROI means you are getting more profit per unit bet. In backtests and reports, ROI is often reported per bet or per series of bets. It's crucial to break this down by segments too (e.g. ROI on favorites vs longshots, ROI in one track vs another) to see where the edge is coming from. Keep in mind the **law of large numbers**: a strategy with 2% ROI over 10,000 bets is more impressive (and likely more reliable) than one with 20% ROI over 50 bets. Practitioners therefore also look at **confidence intervals** of ROI. Nonetheless, **ROI%** gives a quick health check – if it's negative after accounting for commission, the strategy is a non-starter. If positive, how high is it? Many professional approaches in efficient markets shoot for a few percent ROI with high volume. In a guide to NFL modeling, it's noted that ROI (profit on turnover) is perhaps the most crucial summary metric to include when testing a model ³⁰ ³¹.
- **Expected Value (Expected ROI) and Calibration of Edge:** Expected ROI is essentially the average edge per bet. You can estimate this by summing the model's expected profit for each bet and dividing by number of bets. For example, if your model bets at odds that imply 10% edge on average, you'd expect +10% ROI in the long run (before costs). In practice, compare the **model probability vs market probability** for each bet to compute an expected value. Over many bets, the **realized ROI** should converge toward this expected value if the model is sound and well-calibrated. A large discrepancy might indicate mis-calibration or certain biases in execution. As part of evaluation, check if higher model-predicted edges actually yielded higher profits – akin to a calibration curve for ROI. This can be done by bucketizing bets by predicted edge (e.g. 0-5%, 5-10%, 10%+ expected ROI) and seeing if actual returns increase for higher buckets. This helps validate that the model's **value estimates** correlate with outcomes. If not, there may be hidden biases or insufficient data.
- **Turnover and Volume:** **Turnover** refers to the total amount of money wagered. A strategy with a modest edge can still generate substantial profits if it turns over a lot of capital (high frequency of bets). For example, a 2% edge with \$1,000,000 churned through bets is better than a 10% edge on only \$5,000 of bets. Thus, track **number of bets** and total stake. Sometimes ROI is presented alongside **absolute profit** to give context ("Strategy A made 5% ROI over 10,000 bets, yielding \$X

profit"). There is often a trade-off between ROI% and turnover – more frequent betting (including marginal edges) might lower average ROI but increase total profit. One should also compute **Sharpe ratio** or profit volatility metrics in relation to turnover (discussed more below) to assess if the volume is achieved efficiently. In evaluation, ensure that the bet volume you assume is realistic given market liquidity (no point in projecting turnover that the market couldn't support for your stake size – see Execution Realism below).

- **Hit Rate and Win/Loss Distribution:** Hit rate is the percentage of bets that win (for exchange betting, "win" could mean a successful trade or a winning bet). While not directly a measure of profitability (you could have a low hit rate but high ROI if you hit occasional longshots), it's still informative. It gives insight into variance – a low hit rate strategy will have more volatile sequences of results (long losing streaks). Monitor the **distribution of wins and losses**. Are profits coming from a few big wins (which could be luck) or many small edges? Ideally, your strategy isn't relying on extremely rare wins unless those are truly predictable. A strategy might have only 30% winners but still be profitable if the average odds are high enough. **Closing Line Value (CLV)** is another related metric: for each bet, compare the odds you took vs the **closing odds** (final market odds). A consistently positive CLV (you consistently beat the closing price) is a strong indicator your model has predictive value, even if short-term results vary ³¹. Many professionals consider CLV as important as ROI in evaluation – if you're beating the closing market by a margin (after removing commission/vig), it suggests a real edge. If your bets often coincide with odds moves in your favor (e.g. you back at 4.0 and the horse closes at 3.5), that's good; if the opposite happens (you back at 4.0 and it drifts to 5.0 by the off), it might signal overestimation by the model or market information you missed.

- **Drawdown and Risk Metrics: Maximum Drawdown (MDD)** is a key risk metric: it measures the largest peak-to-trough decline of your bankroll during the period. For example, if you started with \$1000, went up to \$1200, then a losing streak brought you down to \$900, that's a \$300 (25%) drawdown from the peak. Even if you recovered later, that 25% is the max drawdown. A strategy that turns a profit in the end might still suffer a gut-wrenching drawdown in the middle. Investors and bettors pay close attention to this – *the lower, the better*. It's common to see guidelines like *aim for MDD under 20%* ³² for a well-tuned strategy (high-frequency quant strategies often have very low drawdowns relative to return, whereas straightforward betting strategies can have higher). Drawdown is related to variance and hit rate. A highly volatile strategy might have a high ROI but also huge swings; whether that's acceptable depends on one's risk tolerance and capital. Evaluate not just the final profit, but how bumpy the ride was. Look at the equity curve of the bankroll and note how long and deep the losing periods were. **Ulcer index** or **Sharpe ratio** (excess return divided by std deviation of returns) are other ways to quantify risk-adjusted performance. Sharpe is borrowed from finance – a Sharpe above 1 or 2 is excellent in financial terms ³², though in betting with small edges it might be lower. The *Pinnacle article on drawdowns* famously shows that even with a positive edge, lengthy losing streaks are probabilistic inevitabilities ³³. So aim to understand, for example, "with a 5% edge and 1000 bets, what's the 95th percentile worst-case drawdown?" – tools like simulation or formulas from the Kelly criterion can estimate this. If the drawdown is uncomfortably high, you may need to bet more conservatively (lower stake sizes). Some experts suggest that a reasonable maximum drawdown to expect even for a good strategy might be 20-30% at some point; if your backtest shows, say, 80% drawdown, it's either too aggressive or not truly an edge. A Medium article on a football value betting strategy noted their top metrics included Sharpe and maximum drawdown, with an aim to keep MDD below 20% by diversifying and refining bets ³⁴.

- **Profit Stability and Statistical Significance:** Check if your results are due to luck or skill. Statistical tests or simply looking at **confidence intervals** of your ROI can help. For instance, Wilson score intervals for win rate, t-tests on average returns, or a bootstrap of your bet outcomes can give a sense of how likely it is to get your observed profit by chance. If you have thousands of bets, small edges can become significant. With only dozens of bets, even a high ROI might not be trustworthy. Also evaluate performance over time (did the model perform consistently each year/quarter, or did one period contribute most of the profit?). A truly robust model should not derive all its success from one golden run – unless that run corresponds to a structural change that you expect to persist (which you'd need to justify).
- **Commission and Fees:** Any evaluation **must account for Betfair's commission and other fees**. Betfair Exchange takes a commission from winning bets (generally ~5% in many jurisdictions, with possible reductions for high volume, or slightly different base rates in some countries) ³⁵. This is on net winnings per market. So your backtest should deduct commission from each win. For example, a bet that returns £100 profit should be counted as £95 profit after 5% commission. Commission effectively reduces your ROI by a factor equal to the commission rate on winners. If you're doing high-turnover strategies like trading (many small wins), commission can be a big drag. The **Premium Charge** is another factor: very successful Betfair traders (who win a lot relative to commission paid) can face additional fees (up to 40% of profits for the top tier). It's complex and usually only kicks in if you are consistently profitable over a long period. In the Betfair model feasibility study, they assumed a **6.5% commission** on all wins to be safe, noting that highly profitable customers might pay higher effective rates ³⁶ ³⁷. So make sure your **profit calculations include commission** – it can turn a +1% edge into negative if not accounted for. In your strategy report, explicitly state "Results are after 5% commission" to be clear. If you plan to operate at high scale, consider modelling the premium charge as well (though that's an advanced consideration for when you're actually beating Betfair handily!).
- **Execution Realism – Slippage and Fill Rate:** Backtests often assume you get the **posted price** and fill for your entire stake. Reality can be different: odds might move by the time your bet goes through, or there might not be enough volume available at the target price, causing you to get partially matched at worse odds. This difference is **slippage** (in trading terms). To account for slippage, you can make a conservative assumption in your backtest: e.g. assume you receive 1 tick worse than the observed price on each bet, or haircut your profit by a certain percentage. If you're trading very short-term moves, slippage and latency become critical – you might need to simulate whether an order would realistically fill before the price changed. For a **bet-and-hold to the finish** strategy, slippage is less about milliseconds and more about ensuring your bet size could be matched at the odds. Betfair provides **volume** at each price – if your stake is larger than the available amount, you will either only get partially matched or you'll also match the next prices (thus effectively taking a worse average price). So, when evaluating, **limit your bet size in the backtest to what the historical order book could support** (or model an algorithm of drip-feeding the order). If your data has full order book depth, you can simulate this precisely. If not, a rule of thumb might be "don't bet more than X% of the traded volume at that moment" to avoid self-impact.
- **Market Impact:** Relatedly, consider that if your strategy is successful and you start wagering larger amounts, you yourself might move the odds, reducing the edge. This isn't a concern in backtesting small stakes, but it is if you plan to scale up. A truly realistic simulation for a big player would need to model how your bets change the market (this is complex and usually not done unless you have

market-making capabilities). For most individual punters at moderate stakes, you can assume you're a price-taker, not a price-maker. Just be aware that any capacity limits will show up as slippage or unfilled bets as you push more money in.

- **Latency and Timing:** If your strategy involves reacting to information (like betting right after a horse's odds drift, or during in-play events), note that in reality there's a delay. In-play betting has a delay (e.g. Betfair imposes ~1 second delay for horse racing bets pre-off, and a few seconds in-play). If your model triggers on something, another algorithm might beat you to the punch. For backtesting, this means if you're using very granular triggers, consider that you might miss some that happened too fast. A practical way is to incorporate a **reaction delay** in the model: e.g. if it sees a price move, assume you only get on the next tick or a second later. This is particularly relevant for high-frequency approaches (less so if your model is based on minutes-long trends or finding value overnight, etc.).
- **Performance under Different Staking Plans:** Thus far we assumed flat stakes or proportional stakes. Part of evaluation is also to see how the strategy performs with different **staking methods**. For example, Kelly criterion staking can maximize growth but also increase risk; level staking is simpler but maybe not optimal. You might simulate a few staking approaches (flat \$100 per bet, Kelly at 1/4 Kelly fraction, etc.) to see the trade-offs in ROI and drawdown. Oftentimes, **Kelly fractions** are used to manage risk – e.g. using half-Kelly to cut volatility. Remember the earlier point: Kelly only makes sense if your probabilities are accurate ²¹; mis-calibration + Kelly can be disastrous. So in evaluation, check how sensitive your outcomes are to staking. If a small change in stake sizing would have caused bankruptcy in the backtest, that's a fragile strategy. Ideally, your edge is robust enough that reasonable staking strategies all yield positive outcomes (with more aggressive ones just giving higher return and higher drawdown).

To summarize, **judge your model like a portfolio**: look at ROI (yield) ³¹, CLV, hit rate, and *especially* risk via drawdowns ³² and variance. Incorporate all relevant costs – commission, and a model of slippage if applicable – so you're not inflating expectations. A good evaluation will tell a complete story: "After 5% commission, the strategy achieved 7% ROI over 5,000 bets, with a longest losing run of 18 bets, a max drawdown of 15% of bankroll (using flat stakes), and a positive CLV of +0.03 (i.e., we beat the closing odds by 3% on average). The profit curve was relatively steady, and even the worst year was profitable." That is the kind of profile you'd want to see before deploying real money. If instead you see huge swings or profits reliant on a handful of bets, you might need to refine the model or accept smaller stakes to mitigate risk.

Ensembles, Stacking, and Gating Strategies

In predictive modeling, **ensembles** (combining multiple models) often outperform single models – and this holds in betting as well. Different models may capture different patterns, and by blending them, you can achieve a more stable and accurate prediction. There are a few ensemble approaches particularly relevant to betting:

- **Model Stacking with Market Odds:** The betting market itself is a powerful predictor (the wisdom of crowds). A common and effective ensemble strategy is to use the **market's implied probabilities as a baseline feature**, and have your model predict the *residual* or adjustment. The idea is that the market odds (like Betfair Starting Price for pre-race, or the live odds at a given moment) already incorporate a ton of information; your model's job is to find where they're slightly off. In the horse

racing study by Betwise, they built a **stacked model that “improves upon betting market odds by stacking our predictions on top of them.”** ¹¹ . In practice, they took their standalone model's prediction and the BSP, and trained a second-stage model to output a refined probability. This yielded a small but consistent edge – essentially the ensemble could beat the market by identifying subtle inefficiencies. You can do this with two levels of models or even within one model (e.g. include the current market odds as an input feature along with all your other features). Many practitioners do this: e.g. in sports like football, one might include the bookmakers' odds or implied win probability as a feature in an ML model that also considers team statistics – this way the model learns to trust the odds most of the time but adjust where the stats suggest an edge. **Caution:** if you include market odds as a feature, ensure you're not “cheating” by using an odds that wouldn't be available at decision time. For pre-play models, BSP is only available at off, so instead use the **current available odds** at the time of bet placement as input. For in-play, you could use the live odds from the previous minute, etc. With that in place, stacking on odds is a very powerful technique; essentially you're piggybacking on the collective intelligence of the market and only adding value where your data or model differs.

- **Diverse Model Ensembles:** You can create an ensemble of different modeling techniques or feature sets. For example, one model might be a gradient boosting machine focusing on form and stats, another might be a neural network ingesting the high-frequency order book data as a time series, and another could be a simple logistic regression using a few intuitive ratios. Each will have strengths and weaknesses. By combining them (through averaging predictions, weighted average, or another meta-model), you often get better overall performance. The variance of the ensemble is lower – an error one model makes might be compensated by another. In the context of horse racing, you might ensemble a **“horse attributes model”** (training, jockey, track, etc.) with a **“market model”** (based on live odds movements). The first might spot a horse that's underrated on paper, the second might catch late market signals – together, they decide if a bet is warranted. Academic research frequently finds ensembles helpful; one paper on football match prediction used an **“evolutionary ensemble classifier”** that combined multiple base models and improved accuracy ³⁸ . In betting, remember accuracy is secondary to profit, but ensemble methods can improve the calibration and stability which feed into profit.

- **Gated Two-Stage Models:** A **gating strategy** means you have one model's output filtered or gated by another criterion or model. A practical betting example: **Model A** predicts win probability for each horse (or expected ROI for a bet). **Model B** (the gate) decides whether to actually place a bet or skip it, based on features and possibly Model A's output. The gate could be a simple rule – e.g. “bet if model overlay > 5% and market spread < 2 ticks and liquidity > X” (ensuring conditions are favorable) – or it could itself be a learned model (classification) that takes in a rich set of features. The goal of gating is to improve **precision** of bets: avoid those situations where the model has an edge on paper but there are other red flags (like very volatile market or low confidence). For instance, one might train a second model on past bets to classify which ones ended up profitable or not, using information like how strong the model's predicted edge was, how much late money came against our position, etc. If that second model can discern “good bets” vs “bad bets” from historical data, you effectively learn a **bet selection filter**. There is evidence this can boost performance. In Season 4 of their strategy, the Systematic Sports team mentioned **recalibrating value and probability thresholds** to be more selective – essentially implementing a stricter gate on what they consider a “value bet” ¹⁰ . By doing so, they reduced noise and focused only on the bets with the highest expected value. This is a form of gating by tightening the criteria for a positive prediction.

- **Ensemble for Classification vs Regression:** If your first-stage model outputs a probability and you use that to bet whenever probability implied edge > 0, you might find that a lot of those low edges aren't actually profitable after transaction costs. A second-stage classifier can learn a threshold or a combination of conditions to decide "bet or no bet". This classifier might effectively learn to ignore edges below, say, 2% or to only take an overlay if certain market indicators concur. Another example of gating: maybe your model identifies value on longshots often, but you notice the market is very thin there (leading to slippage). A gate model could learn to pass on very illiquid longshots even if they seem to have value, to improve real ROI. The beauty of a learned gate is it can weigh multiple factors. It's essentially learning from the model's mistakes: feed it features of bets (including model's predicted edge, market conditions, etc.) and label 1 for profitable bet, 0 for unprofitable (after the fact). It might identify patterns (perhaps many "edges" on outsiders were false positives, etc.) and thus refine future selections.
- **Blending Predictions for Final Output:** If not using a hard gate, you can also blend model outputs for a final probability. For instance, **blending an implied probability from market with your model's probability:** a simple way is average them, or a weighted average giving more weight to the market for lower confidence situations and more weight to the model when certain conditions are met. More sophisticated is to train a meta-model (like a linear regression or neural net) that takes as input `[model1_prob, model2_prob, ..., market_prob]` and outputs a final probability. This meta-model will automatically learn how much to trust each input. It might learn, say, to trust the market heavily except in cases where model1 has a significantly different opinion *and* model1's confidence is high. This is essentially what stacking achieves. The Betwise team's stacked model effectively did this and demonstrated a small edge over pure market odds ¹¹. In their case, if their model had no insight, the stacked outcome would just mirror the market; but where the model had insight, the stacked outcome shifted slightly, yielding an edge when the model was right.
- **Ensembles for Risk Management:** Sometimes ensembles are used not just for accuracy, but to stabilize risk. For example, running two independent strategies: one might be mean-reversion based (betting that drifters will swing back), another momentum based (betting that steamers will continue). They might have uncorrelated performance, so combining them smooths the overall equity curve. In a machine learning context, you could train models on different subsets of data or different targets and then allocate capital to each. The **portfolio approach** to betting says that combining diverse edges is beneficial as long as they're not perfectly correlated. If you have multiple models with edges on different race types or different markets, use them all and treat it like a multi-asset portfolio – this is an ensemble at the strategy level rather than prediction level.

In conclusion, **don't rely on a single model or signal**. Use the collective power of ensembles: incorporate the market opinion as a baseline, and layer your insights on top. Use multiple models to capture various angles (one might catch something another misses). And consider a second-stage filter to weed out bets that look good on paper but likely won't materialize into profit. Both academic research and real betting practitioners have gravitated to these approaches in recent years, finding that an ensemble or stacked model often yields **more reliable and profitable predictions** than any individual model alone ¹¹ ¹⁰. By providing a safety net and combining strengths, ensembles and gated models can significantly improve the **consistency and returns** of a betting strategy.

Sources: Recent literature and community resources have extensive discussions on these topics. For instance, Gonçalves *et al.* (2025) demonstrate short-term price prediction in Betfair horse markets using

deep networks ⁹, Walsh & Joshi (2024) highlight the importance of calibration for profitability ⁷, and Betfair's own data science blog tutorials (e.g. Tom Bishop's analysis on market formation) illustrate how to engineer features like overround, spread, and traded volume ¹ ³. Practitioner write-ups, such as those on the Betfair Hub and analytics blogs, show practical applications: e.g. walk-forward validation for NFL models ¹⁴, controlling drawdowns in a football betting portfolio ³⁴, and identifying value bets by comparing model probabilities to odds ⁶. All these sources reinforce the strategies discussed here and provide further details for interested readers.

¹ ² ³ ⁵ Do they know? Analysing Betfair market formation & market movements - The Automation Hub

<https://betfair-datascientists.github.io/tutorials/analysingAndPredictingMarketMovements/>

⁴ Strategies for Maximizing Profits on Betting Exchanges

<https://sports.ocsoftware.com/strategies-for-maximizing-profits-on-betting-exchanges/>

⁶ Using Probability Models to Improve Betting Accuracy - SDLC Corp

<https://sdlccorp.com/post/using-probability-models-to-improve-betting-accuracy/>

⁷ ⁸ ²⁰ ²¹ Machine learning for sports betting: Should model selection be based on accuracy or calibration? - ScienceDirect

<https://www.sciencedirect.com/science/article/pii/S266682702400015X>

⁹ [2510.16008] Convolutional Attention in Betting Exchange Markets

<https://arxiv.org/abs/2510.16008>

¹⁰ ³² ³⁴ Our Five Football Betting Strategy Secrets | by Estèphe | Systematic Sports | Medium

<https://medium.com/systematic-sports/five-secrets-of-our-football-betting-strategy-e3c188d5c335>

¹¹ ¹⁵ ²⁸ ²⁹ ³⁵ ³⁶ ³⁷ Feasibility of building a profitable data-driven betting system on horse races

https://www.betwise.co.uk/publications/Feasibility_of_building_a_profitable_data_driven_model_for_betting_on_horseracing.pdf

¹² ¹³ ¹⁴ ¹⁶ ¹⁸ ¹⁹ ³⁰ ³¹ Build a Winning NFL Betting Model

<https://nxtbets.com/winning-nfl-betting-model/>

¹⁷ ²⁷ How to Calibrate Probabilities for Imbalanced Classification - MachineLearningMastery.com

<https://machinelearningmastery.com/probability-calibration-for-imbalanced-classification/>

²² ²³ lightgbm.LGBMClassifier — LightGBM 4.6.0.99 documentation

<https://lightgbm.readthedocs.io/en/latest/pythonapi/lightgbm.LGBMClassifier.html>

²⁴ ²⁵ ²⁶ New Feature - Adjusted Probability when using scale_pos_weight · Issue #863 · dmlc/xgboost · GitHub

<https://github.com/dmlc/xgboost/issues/863>

³³ What are drawdowns and how do you manage them? - Pinnacle

<https://www.pinnacle.com/betting-resources/en/educational/what-are-drawdowns-and-how-do-you-manage-them/5wr2bjk223g5zfsc>

³⁸ [PDF] Predicting football results with an evolutionary ensemble classifier

https://vu-business-analytics.github.io/internship-office/reports/report-hoekstra_vincent.pdf